

CS-3510-B Problem Set 6

Elijah Martin McCauley

TOTAL POINTS

45 / 50

QUESTION 1

1 Problem 1 20 / 25

- 0 pts Correct

Table Definition (5 pts)

- 0 pts Correct (no penalty)
- 1 pts Minor mistake
- 3 pts Mostly incorrect table definition.
- 5 pts Table Definition Missing/Completely incorrect

Base Cases and Recurrence (10 pts)

- 1 pts Minor mistake in base case/ Missing one base case
- 2 pts Base Case Missing/Completely Incorrect
- 1 pts Minor mistake in recurrence
- 2 pts Recurrence does not align with the table defined.
- 2 pts Explanation of correctness

Missing/Incorrect

- 6 pts Recurrence Missing/Completely incorrect
- 4 pts Major mistake in recurrence relation
- 10 pts Incorrect Solution/ Chain-Matrix

Approach not used

Runtime Analysis (5 pts)

- 1 pts Minor mistake
- 2 pts Correct runtime with no/incorrect analysis.

- 3 pts Correct runtime and analysis but suboptimal solution.

✓ - 5 pts *Missing/Incorrect Runtime and analysis*

Pseudocode (5 pts)

- 1 pts Minor Error (typo, missing an index, etc)
- 2 pts Code does not use the recurrence presented/ does not align with the table defined.
- 3 pts Multiple minor errors or one major error that breaks correctness.
- 4 pts Mostly incorrect/ dynamic programming isn't used
- 3 pts Description given instead of code
- 5 pts Pseudo-code Missing
- 10 pts Inefficient Solution
- 25 pts Missing Answer

QUESTION 2

2 Problem 2 25 / 25

✓ - 0 pts *Correct*

Table Definition (5 pts)

- 0 pts Dimensions missing (no penalty)
- 1 pts Minor mistake
- 3 pts Mostly incorrect table definition.
- 5 pts Table Definition Missing/Completely incorrect

Base Cases and Recurrence (10 pts)

- 1 pts Minor mistake in base case/ Missing one

base case

- **2 pts** Base Case Missing/Completely Incorrect
- **1 pts** Minor mistake in recurrence
- **2 pts** Recurrence does not align with the table

defined.

- **2 pts** Explanation of correctness Missing
- **6 pts** Recurrence Missing/Completely incorrect
- **4 pts** Major mistake in recurrence relation

Runtime Analysis (5 pts)

- **1 pts** Minor mistake
- **2 pts** Correct runtime with no/incorrect

analysis.

- **3 pts** Correct runtime and analysis but suboptimal solution.

- **5 pts** Missing/Incorrect Runtime and analysis

Pseudocode (5 pts)

- **1 pts** Minor Error (typo, missing an index, etc)
- **2 pts** Code does not use the recurrence

presented/ does not align with the table defined.

- **3 pts** Multiple minor errors or one major error that breaks correctness.

- **4 pts** Mostly incorrect/ dynamic programming isn't used

- **5 pts** Pseudo-code Missing

- **10 pts** Inefficient solution

- **25 pts** Missing Answer

Problem 1

(25 points)

Our wonderful TAs, Joseph and Stephen, are playing a game! They have a list of cards L where each card has a known value $v_1, \dots, v_n$. The cards can be of any suit, and the values might be repeated as there are four cards of the same values in a deck. Joseph and Stephen then take turns picking a card and removing it from the list, receiving its value as points. They're only allowed to take cards from the front or back of the list i.e. the front position or the last position in the list L . Design a dynamic programming algorithm to compute the maximum value that Joseph can achieve assuming they both play optimally (that is, trying to maximize their own score).

Assume that $\text{len}(L)$ is an even positive integer. Joseph will always start the game and they can only pick one card at a time.

Example:

Input: $L = [3, 5, 1, 5, 8, 11]$

Output: 21

Hint: Think of an approach using Matrix chain multiplication. Greedy and non-DP solutions won't receive any points.

- (a) Define the entries of your table in words. E.g. $T(i, j)$ is ...

Solution: The entries of our table $T[i][j]$ equals the maximum value that Joseph can get when given the cards $L[i]$ through $L[j]$. For example, $T[1][2] =$ the best score he can get given only card 1 and 2.

- (b) State recurrence for entries of the table in terms of smaller subproblems. Briefly explain in words why it is correct. Remember to state your base case(s) as well.

Solution: The base cases are when $i = j$ which means there is only one card to pick, so $T[i][i] = L[i]$. Additionally, when only given two cards, Joseph picks the maximum of the two, so when $j - i = 1$, $T[i][j] = \max(L[i], L[j])$. This has to be done because our relation needs to access $T[i][j-2]$, so we need to make sure this relation doesn't start until $j > 1$ to avoid index out of bounds. Our relation for $j - i > 1$, $T[i, j] = \max(L[i] + \min(T[i+2][j], T[i+1][j-1]), L[j] + \min(T[i][j-2], T[i+1][j-1]))$. So basically, we want to take the edge card which maximizes our value while minimizing their value at the same time.

- (c) Write **pseudocode** for your algorithm to solve this problem.

Solution:

```
def cardgame(L){
    L.length - 1 = n
    allocate T[0...n][0...n]
    for i = 0...n:
```

```

        T[i][i] = L[i]

    for diff = 1...n:
        for i = 0...n-diff:
            j = i + diff
            if(j - i = 1):
                T[i][j] = max(L[i], L[j])
            else:
                T[i, j] = max(L[i] + min(T[i+2][j], T[i+1][j-1]),
                               L[j] + min(T[i][j-2], T[i+1][j-1]))

    return T[0][n]
}

```

(d) Analyze the running time of your algorithm.

Solution: This algorithm runs in $\mathcal{O}(n \log(n))$ time because while it is a nested for loop, the inner for loop is shortened as the value of the outside one gets larger.

1 Problem 1 20 / 25

- 0 pts Correct

Table Definition (5 pts)

- 0 pts Correct (no penalty)
- 1 pts Minor mistake
- 3 pts Mostly incorrect table definition.
- 5 pts Table Definition Missing/Completely incorrect

Base Cases and Recurrence (10 pts)

- 1 pts Minor mistake in base case/ Missing one base case
- 2 pts Base Case Missing/Completely Incorrect
- 1 pts Minor mistake in recurrence
- 2 pts Recurrence does not align with the table defined.
- 2 pts Explanation of correctness Missing/Incorrect
- 6 pts Recurrence Missing/Completely incorrect
- 4 pts Major mistake in recurrence relation
- 10 pts Incorrect Solution/ Chain-Matrix Approach not used

Runtime Analysis (5 pts)

- 1 pts Minor mistake
- 2 pts Correct runtime with no/incorrect analysis.
- 3 pts Correct runtime and analysis but suboptimal solution.

✓ - 5 pts *Missing/Incorrect Runtime and analysis*

Pseudocode (5 pts)

- 1 pts Minor Error (typo, missing an index, etc)
- 2 pts Code does not use the recurrence presented/ does not align with the table defined.
- 3 pts Multiple minor errors or one major error that breaks correctness.
- 4 pts Mostly incorrect/ dynamic programming isn't used
- 3 pts Description given instead of code
- 5 pts Pseudo-code Missing
- 10 pts Inefficient Solution
- 25 pts Missing Answer

Problem 2

(25 points)

Assume you are taking CS 3510 Exam 3 and have exactly M minutes to answer N questions. You know the point value p_i for each question and have determined that each question will take you t_i time to answer. You also cannot answer more than K questions on your exam. Design a dynamic programming algorithm to determine if you can achieve at least 70% on the exam. Assume that you will answer each question you attempt correctly.

Hint: Start by ignoring the K constraint, then try to add it in as a separate dimension.

- (a) Define the entries of your table in words. E.g. $T(i)$ or $T(i, j)$ is ...

Solution: The entries of our table $T[i][j][k]$, are the maximum number of points you can earn in j minutes with k questions answered using problems 1...i.

- (b) State recurrence for entries of the table in terms of smaller subproblems. Briefly explain in words why it is correct. Remember to state your base case(s) as well.

Solution: The base cases are when i , j , or k are 0 the entry is zero because we would have either 0 time, can answer 0 questions, or are given 0 problems, so $T[0][j][k] = 0$, $T[i][0][k] = 0$, and $T[i][j][0] = 0$. The recurrence relation is $T[i][j][k] = \max(T[i-1][j][k], T[i-1][j-t[i]][k-1] + p[i])$. Where $p[i]$ is the point value for question i , and $t[i]$ is the time it takes to do problem i .

- (c) Write **pseudocode** for your algorithm to solve this problem.

Solution: We change the recurrence relation slightly because of the way we are looping, since our loops are starting at 1, but our P and T index starting at 0, we need to subtract 1 from i in order to access the correct index in the P and T tables.

```
def maxtestpoints(M, N, P, T, K) {
    allocate dp[0...N][0...M][0...K]
    dp[0...N][0...M][0...K] = 0
    for j = 1...M:
        for k = 1...K:
            for i = 1...N:
                if j >= T[i-1]:
                    dp[i][j][k] = max(dp[i-1][j][k],
                                         dp[i-1][j-T[i-1]][k-1] + P[i-1])
                else:
                    dp[i][j][k] = dp[i-1][j][k]

    sum = 0
    for x in 0...N:
        sum += P[x]
```

```
        return dp[N][M][K]/sum >= .7  
    }
```

(d) Analyze the running time of your algorithm.

Solution: The running time for this algorithm would be $\mathcal{O}(NMK)$ because we run nested for loops of size N, M, and K, while the rest of the work we do is negligible in time.

2 Problem 2 25 / 25

✓ - **0 pts** Correct

Table Definition (5 pts)

- **0 pts** Dimensions missing (no penalty)
- **1 pts** Minor mistake
- **3 pts** Mostly incorrect table definition.
- **5 pts** Table Definition Missing/Completely incorrect

Base Cases and Recurrence (10 pts)

- **1 pts** Minor mistake in base case/ Missing one base case
- **2 pts** Base Case Missing/Completely Incorrect
- **1 pts** Minor mistake in recurrence
- **2 pts** Recurrence does not align with the table defined.
- **2 pts** Explanation of correctness Missing
- **6 pts** Recurrence Missing/Completely incorrect
- **4 pts** Major mistake in recurrence relation

Runtime Analysis (5 pts)

- **1 pts** Minor mistake
- **2 pts** Correct runtime with no/incorrect analysis.
- **3 pts** Correct runtime and analysis but suboptimal solution.
- **5 pts** Missing/Incorrect Runtime and analysis

Pseudocode (5 pts)

- **1 pts** Minor Error (typo, missing an index, etc)
- **2 pts** Code does not use the recurrence presented/ does not align with the table defined.
- **3 pts** Multiple minor errors or one major error that breaks correctness.
- **4 pts** Mostly incorrect/ dynamic programming isn't used
- **5 pts** Pseudo-code Missing
- **10 pts** Inefficient solution
- **25 pts** Missing Answer

Problem Set 6: Dynamic Programming II

Prof. Abraham Ladha

Due: 03/13/2023 11:59pm

- Please **TYPE** your solutions using Latex or any other software. Handwritten solutions *won't* be accepted.
- Please write pseudo-code for this problem set, unless otherwise stated to write the algorithm in English.
- Fastest (and correct) solutions are worth more credit.

Problem 3 (*Optional, but highly recommended for the exam*)

Note: We won't grade this problem, but it's highly recommended that you solve the problem yourself. This question uses the Knapsack approach.

Due to a shortage of jute, you have been provided only one piece of mini-rope each of length $s_1, \dots, s_n$. You wish to create a long rope of integer length S by tying some of the ropes given. Design a dynamic programming algorithm that returns if it is possible to create a long rope of length exactly S using ropes of lengths $s_1, \dots, s_n$.

You are allowed to use each rope s_i only once and assume the knot is of length 0, i.e., the knot doesn't consume any of the lengths of either of the rope pieces.

Examples:

Input: $s_1 = 5, s_2 = 7, s_3 = 9, S = 17$

Output: False

Input: $s_1 = 5, s_2 = 7, s_3 = 9, S = 14$

Output: True

Explanation: Use one rope of length s_1 , and one rope of length s_3 $5 + 9 = 14$.

- (a) Define the entries of your table in words. E.g. $T(i)$ or $T(i, j)$ is ...
- (b) State recurrence for entries of the table in terms of smaller subproblems. Briefly explain in words why it is correct. Remember to state your base case(s) as well.
- (c) Write **pseudocode** for your algorithm to solve this problem.
- (d) Analyze the running time of your algorithm.

Problem 4 (*Optional, but highly recommended for the exam*)

Note: We won't grade this problem, but it's highly recommended that you solve the problem yourself. This question uses the Matrix chain Multiplication approach.

We call a sequence of alphabetical characters symmetric if it reads the same backward as it does forwards. An example of a symmetric sequence is a, b, c, b, a . An example of a sequence that is not symmetric is d, a, c, d, a .

Design a dynamic programming algorithm that takes as input a sequence of alphabetical characters A of length n and returns the length of the longest symmetric subsequence of A . Note that this subsequence is not necessarily contiguous.

Example:

Input: $A = a, b, c, d, c, f, a$

Output: 4

Explanation: a, c, a is the longest symmetric subsequence of A .

Input: $A = a, b, c, d, f$

Output: 0

Explanation: a, c, a is the longest symmetric subsequence of A .

- (a) Define the entries of your table in words. E.g. $T(i)$ or $T(i, j)$ is ...
- (b) State recurrence for entries of the table in terms of smaller subproblems. Briefly explain in words why it is correct. Remember to state your base case(s) as well.
- (c) Write **pseudocode** for your algorithm to solve this problem.
- (d) Analyze the running time of your algorithm.